RBE 595 - Homework 6

Fall 2026

THIS HOMEWORK IS ONLY FOR THE STUDENTS ENROLLED TO THE GRADUATE COURSE RBE 595. IF YOU ARE ENROLLED TO THE UNDERGRADUATE CLASS, PLEASE CHECK THE GROUP ASSIGNMENT.

This assignment should be completed and submitted individually, i.e., you will not be solving it as a team.

For this assignment, you will submit a project report containing the deliverables listed below.

In this project, you will implement a grasping pipeline that uses images from RGB-D cameras, synthesizes grasps for objects in the scene, and commands the robot to pick the objects and place them at target locations.

You will accomplish these goals using point-cloud processing tools. One such tool is the Point Cloud Library (PCL), which provides high-level implementations of RANSAC, clustering, filtering, and other point-cloud processing methods. You will use these tools to preprocess the data, segment the target objects, and search for stable grasps.

You will implement and test these techniques in the Gazebo simulation environment.

Simulation Setup

Pull the latest version of the simulation from https://github.com/berkcalli/rbe4540-gazebo-sim.git. Inside the main src folder, you will find several README files. Open README_PC_SETUP.md and follow the instructions to install PCL and the supporting tools. Then build your workspace and source its setup file.

In the current version, the pc_processing_template.py node receives a point cloud from the camera and transforms it into the robot base frame. The process_cloud and estimate_grasp functions are intentionally left empty and will be completed by you. You can visualize the input and processed point clouds in RViz.

There are three cameras in the simulated scene: two fixed RGB-D cameras positioned on opposite sides of the table and one eye-in-hand camera attached to the robot. You will use the two fixed RGB-D cameras in this assignment.

Review the other README files in the src folder as well. They contain example implementations of several PCL functions that may be useful throughout the assignment.

Important note: Write your code as modularly and parametrically as possible. Object positions may need to change depending on the robot's reachable workspace. If your implementation is parameterized, these changes should require only a small number of parameter updates. Avoid hard-coding values throughout your program.

Step 1 - Point Cloud Preprocessing

The goal of this step is to segment the target objects and prepare their point clouds for grasp synthesis.

Step 1.1 - Merge the point clouds from both cameras

If you are modifying the provided template code directly, create a second subscriber and callback function to receive the point cloud from the second camera. The point cloud from the first camera is already transformed into the robot base frame. Apply an equivalent transformation to the point cloud from the second camera so that both clouds are represented in the same coordinate frame. Then merge the two point clouds.

After merging them, downsample the combined cloud so that overlapping measurements are not unnecessarily duplicated. Do not downsample too aggressively, as this may negatively affect plane segmentation, normal estimation, and grasp detection.

> **Deliverable:** Display the merged and downsampled point cloud in RViz, take a screenshot, and include it in your report.

Step 1.2 - Filter the workspace

Filter the point cloud, for example using a pass-through filter, so that you retain only the region of the table that contains the objects of interest. Remember that the point cloud is already represented in the robot base frame.

> **Deliverable:** Display the filtered point cloud in RViz, take a screenshot, and include it in your report.

Step 1.3 - Segment the table plane

Apply RANSAC to segment the table plane. Example implementations for plane extraction are provided in the README files.

> **Deliverable:** Display the result in RViz, take a screenshot, and include it in your report.

Step 1.4 - Segment the objects

Remove the points that belong to the table plane so that the remaining points correspond to the objects.

> **Deliverable:** Display the segmented object points in RViz, take a screenshot, and include it in your report.

Step 1.5 - Cluster the objects

Use a clustering algorithm to separate the points belonging to individual objects.

> **Deliverable:** Display the point cloud for each object individually in RViz, take screenshots, and include them in your report.

Step 2 - Grasp Synthesis

In this step, you will use surface normals and the object centroid to identify a stable grasp.

Step 2.1 - Calculate the object centroid

Calculate the centroid of each object point cloud, for example by averaging the point coordinates.

> **Deliverable:** Display the centroid together with the corresponding object point cloud.

Step 2.2 - Estimate surface normals

Estimate the surface normals for an individual object. Refer to the provided README examples. In the example that orients normals toward a specified viewpoint, use the object centroid as the reference point so that the selected normal direction approximately points toward the object center.

Step 2.3 - Select for feasible grasp-point pairs

Search through the candidate points for pairs of surface normals that satisfy the stable-grasp threshold using the approximation discussed in class. Among the feasible grasp-point pairs, select the pair whose grasp points are closest to the object centroid.

> **Deliverable:** Display the selected grasp points together with the object point cloud.

Step 3 - Calculate the Grasp Orientation (Approach Vector)

The selected grasp points can be approached from multiple directions. To determine an approach vector, compute the principal components of the object point cloud using PCA and choose an approach direction that is orthogonal to the object's major axis.

> **Deliverable:** Display the selected grasp points and the approach direction together with the object point cloud.

Step 4 - Execute the Grasp

Command the robot to move to the grasp pose, grasp the object, and lift it. You may first define a pre-grasp pose located along the grasp approach direction, with the gripper still clear of the object. From the pre-grasp pose, move along the approach vector until the grasp points are reached, then close the gripper and lift the object.

Using a pre-grasp pose helps reduce the chance of colliding with or knocking over the object while moving directly to the final grasp pose.

> **Deliverable:** Submit a video showing the robot moving to the grasp pose, executing the grasp, and lifting the object.

Submission

- A project report containing all screenshots and intermediate results requested above.
- Your complete implementation/code.
- A video showing the final grasp execution.

Bonus Implementation (up to 30 bonus points)

You may receive bonus points by implementing an alternative grasping approach in addition to the method described above. One option is a top-surface-based grasp-synthesis method: after segmenting each object, further segment its top surface and apply a 2D grasp-synthesis algorithm.

Useful PCL-style operations

```
# Choose workspace bounds and crop (pcl::CropBox).
cloud = cloud.crop_box(minimum=[...], maximum=[...])

# Find the table with pcl::SACSegmentation (RANSAC).
inliers, plane = cloud.segment_plane(distance_threshold=..., max_iterations=...)

# Separate objects with pcl::EuclideanClusterExtraction.
clusters = cloud.euclidean_clusters(tolerance=..., min_size=..., max_size=...)

# Estimate normals with pcl::NormalEstimation if useful.
normals = target.estimate_normals(radius=..., viewpoint=[...])
```